

Stop Overprovisioning for Application Startup



DevRel Engineer @ Akamas

Tech Lead @ CNCF TAG DevEx, Kubernetes Release Team



Casto Graziano



castograziano.com

Before getting started

Why we should care about application startup?

75% use K8s.

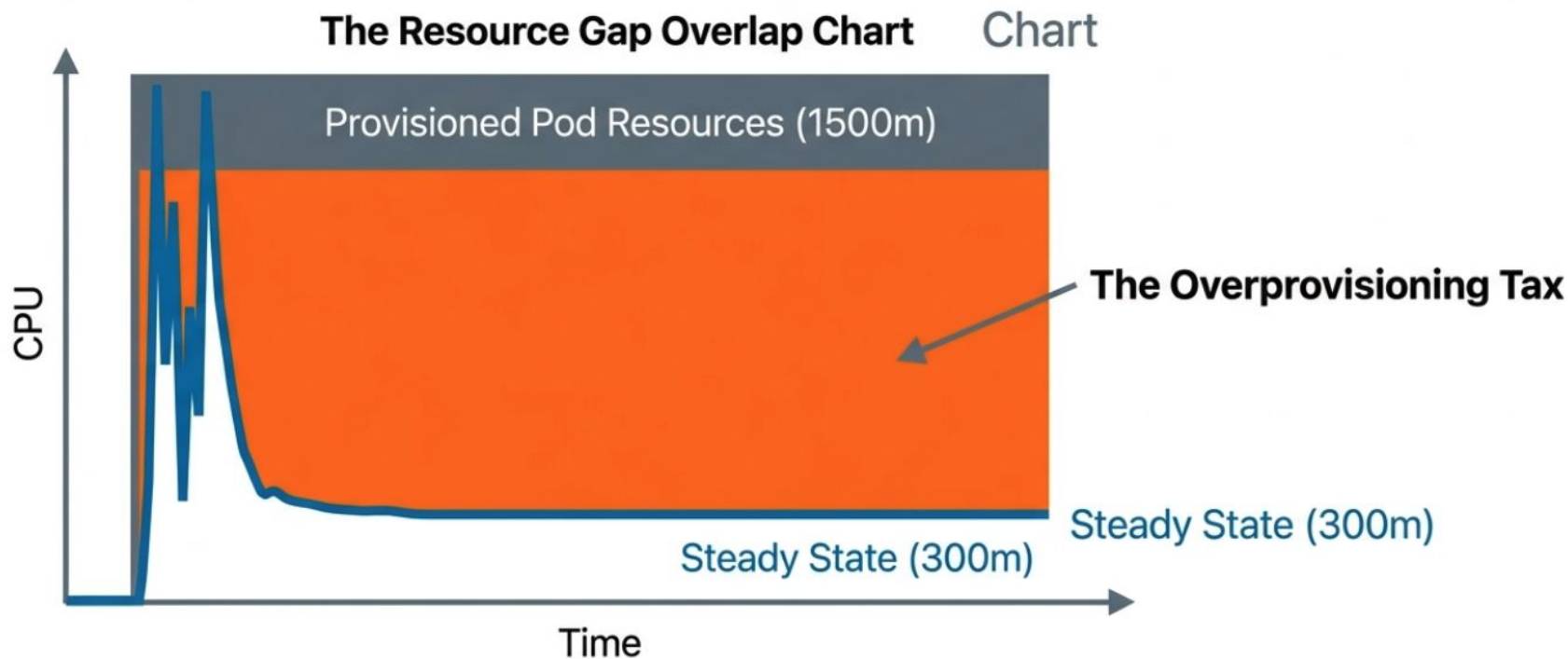
JVM *is the dominant runtime*
(Java/Kotlin)



State of Cloud Native Optimization 2026

The Problem

How your Pods looks like (most likely)



The Problem

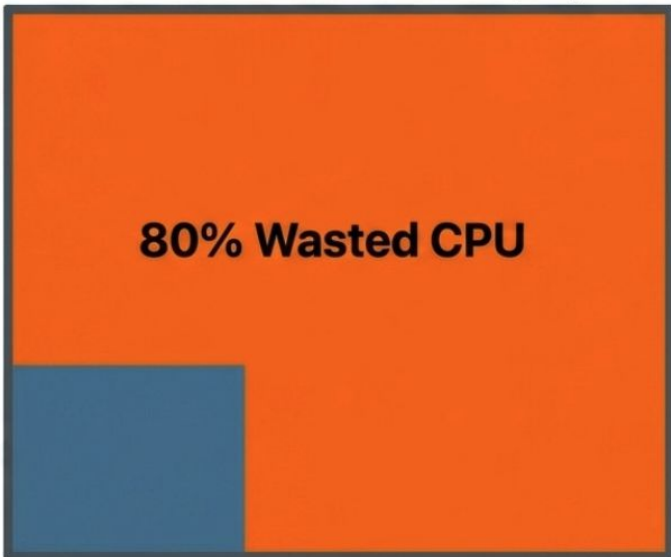
What's the impact at scale?

Proportional Area Chart

Total Provisioned CPU: 1500m per pod

80% Wasted CPU

Steady State
Need: 300m



Scenario A:

10 services x 5 replicas = **60 vCPU** wasted

Scenario B:

50 services x 5 replicas = **300 vCPU** wasted

Option 1:

Rightsize the Pod after the warm up



Option 2:

Reduce the spikes on the warm up phase



How can we solve this problem?

In Place Pod Resize (IPPR)

How to change req/limit without restarting the Pod

Kubernetes Blog

Kubernetes 1.35: In-Place Pod Resize Graduates to Stable

By **Natasha Sarkar (Google)** | Friday, December 19, 2025

This release marks a major step: more than 6 years after its initial conception, the **In-Place Pod Resize** feature (also known as In-Place Pod Vertical Scaling), first introduced as alpha in Kubernetes v1.27, and graduated to beta in Kubernetes v1.33, is now **stable (GA)** in Kubernetes 1.35!

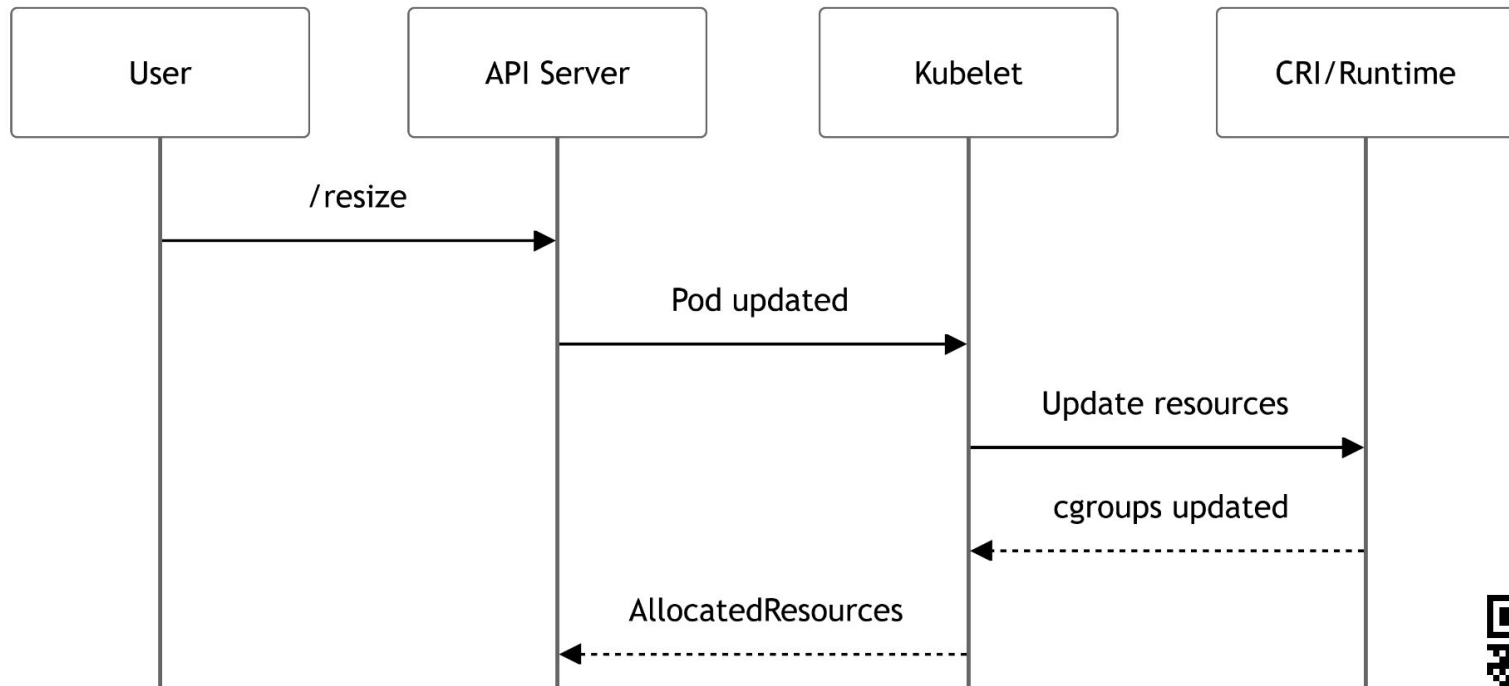
This graduation is a major milestone for improving resource efficiency and flexibility for workloads running on Kubernetes.



IPPR Feature Blog (v1.35)

In Place Pod Resize (IPPR)

How it works



In Place Pod Resize (IPPR)

So everything is mutable now?



CPU



Memory



JDK / JDK-8359211

Automatic Heap Sizing for G1

Submitted

Details

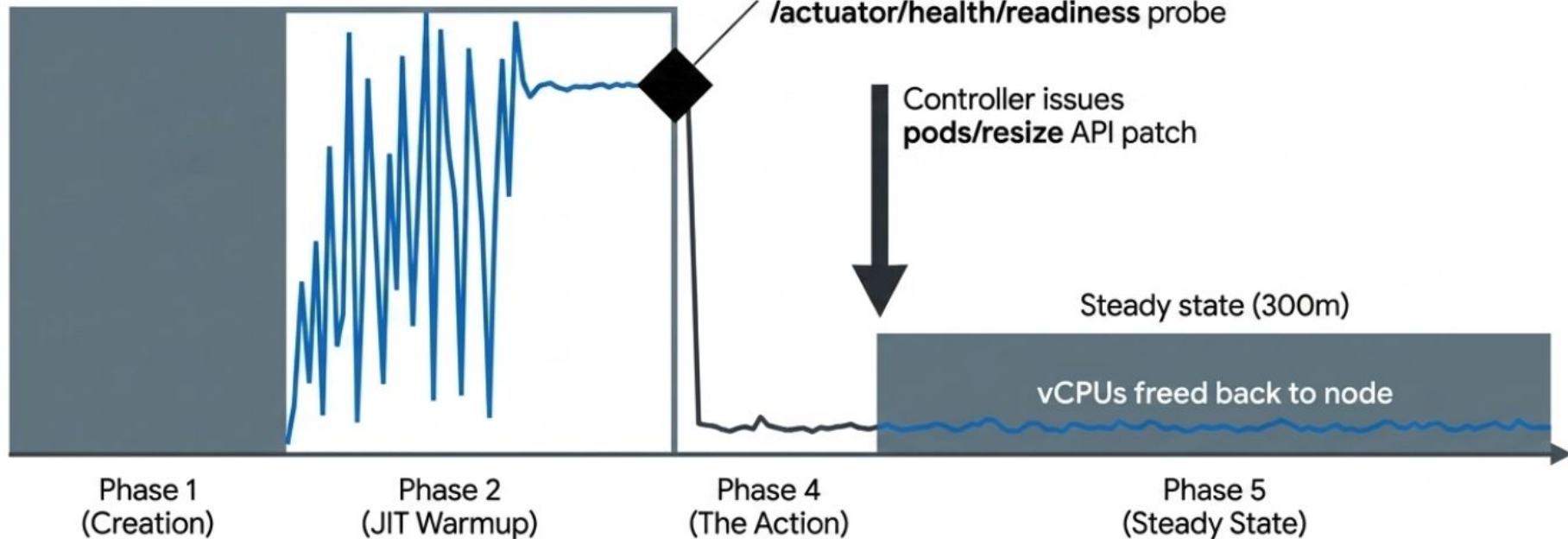
Type:	+ JEP	Resolution:	Unresolved
Priority:	P5	Fix Version/s:	None
Component/s:	hotspot		
Labels:	amazon-interest gc-g1 gc-g1-heap-resizing google-interest		
Author:	Thomas Schatzl		
JEP Type:	Feature		
Exposure:	Open		
Subcomponent:	gc		
Scope:	Implementation		
Discussion:	hotspot dash gc dash dev at openjdk dot org		
Effort:	L		
Duration:	L		



Solution

How can we use IPPR?

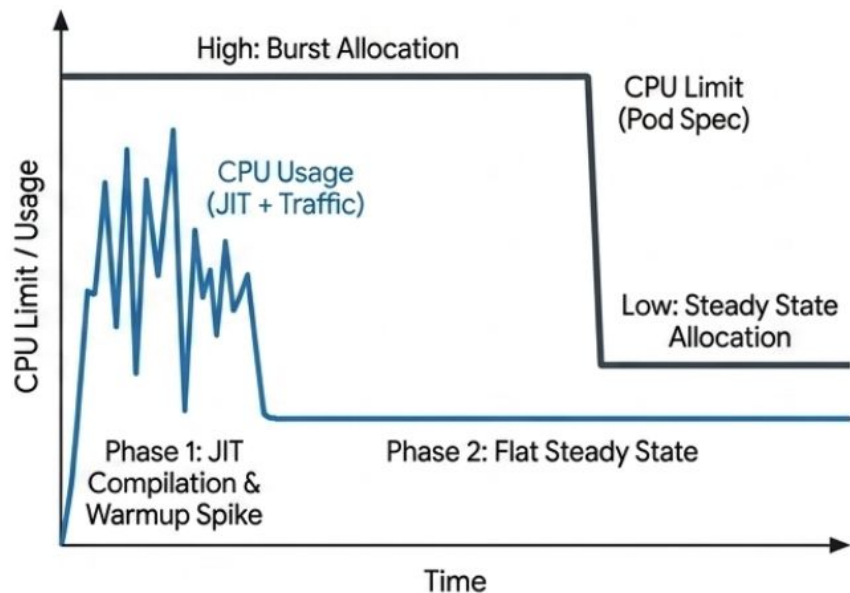
Pod starts with Burst CPU (1500m)



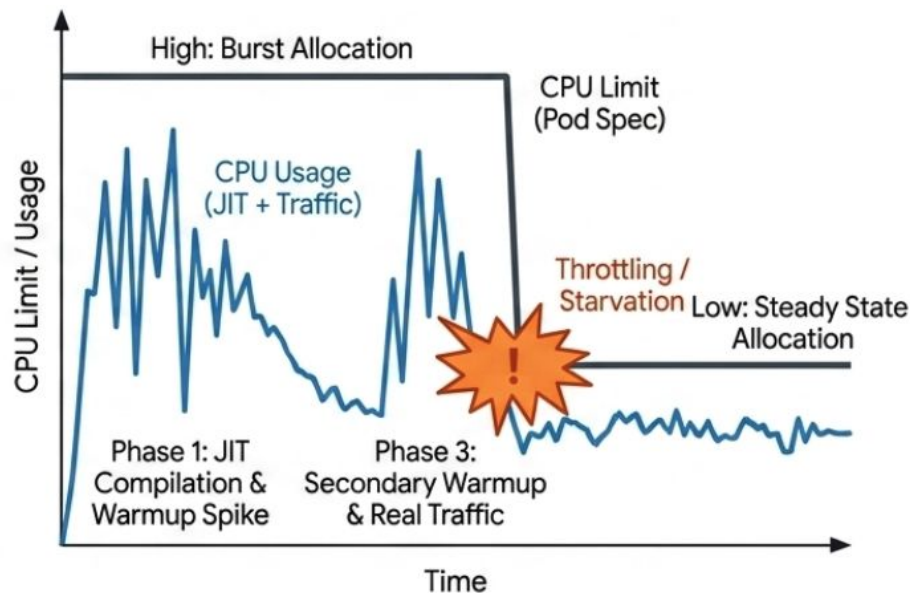
Solution

How can we use IPPR?

Optimal Timing

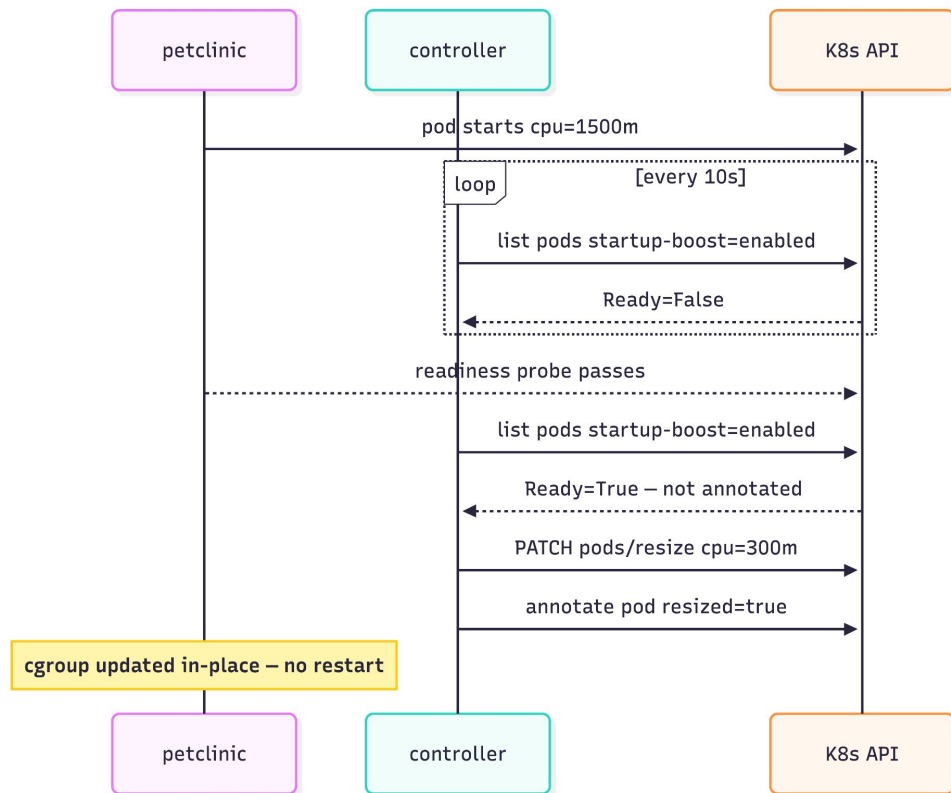


Premature Shrinkage



Possible Implementations

Single Deployment - Controller



Give me all pods with the label *startup-boost=enabled*. For each one: is it Ready? Has it already been resized? No? Then I call the Kubernetes API to drop the CPU to 300m, and I write an annotation on the pod so I remember I already handled it.

Possible Implementations

Single Deployment - Controller

```
metadata:
  labels:
    app.kubernetes.io/name: petclinic
    startup-boost: "enabled"
resources:
  requests:
    cpu: "1500m"
    memory: "512Mi"
  limits:
    cpu: "1500m"
    memory: "1Gi"
```

petclinic-deployment.yaml



```
containers:
- name: controller
  image: bitnami/kubectl:latest
  command: ["/bin/sh", "/hooks/controller.sh"]
  volumeMounts:
    - name: hooks
      mountPath: /hooks
  env:
    - name: CONTROLLER_NAMESPACE
      valueFrom:
        fieldRef:
          fieldPath: metadata.namespace
    - name: STEADY_CPU
      value: "300m"
    - name: STEADY_MEMORY
      value: "512Mi"
    - name: CONTAINER_NAME
      value: "petclinic"
    - name: POLL_INTERVAL
      value: "10"
```

resizer-controller-deployment.yaml

Possible Implementations

Single Deployment - Controller



```
resizePolicy:  
  - resourceName: cpu  
    restartPolicy: NotRequired
```

petclinic-deployment.yaml

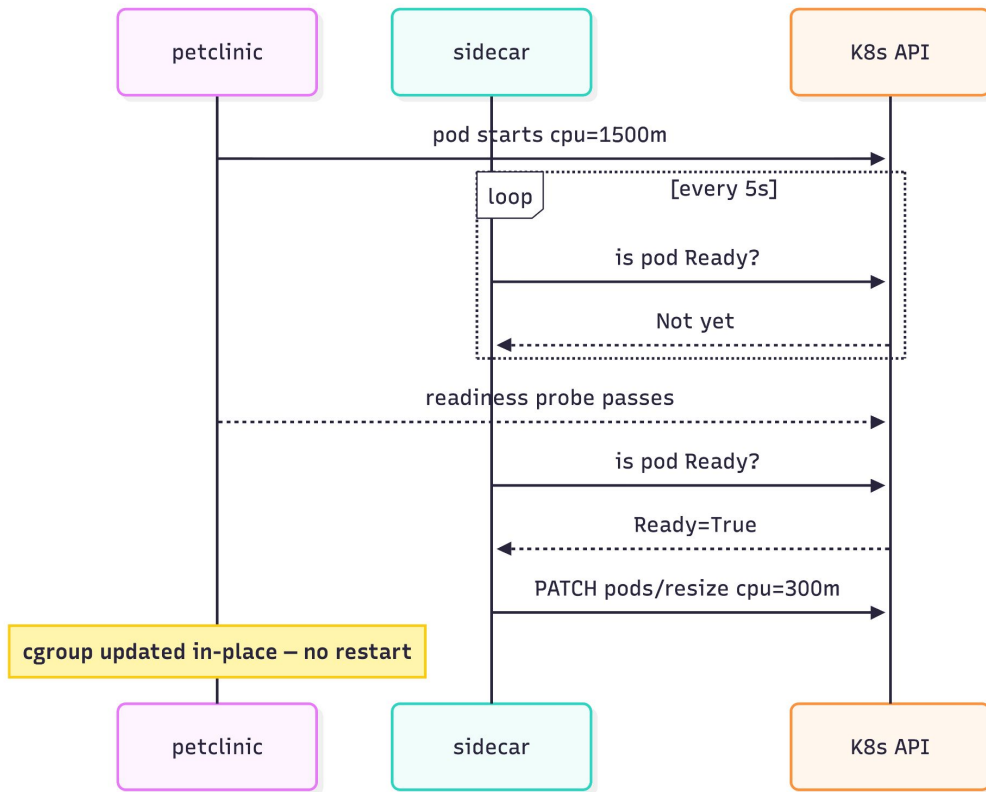


```
kubectl patch pod $POD \  
  --subresource=resize \  
  --type=strategic \  
  -p '{"spec":{"containers":[{"name":"petclinic","resources":{"requests":{"cpu":"300m"}}}]}}'
```

controller.sh

Possible Implementations

Sidecar Pattern



I am running *inside the same pod* as the application. I watch its readiness endpoint every 5 seconds, and the moment it says it is ready I call the Kubernetes API to drop the CPU to 300m. Then I go to sleep — my job is done.

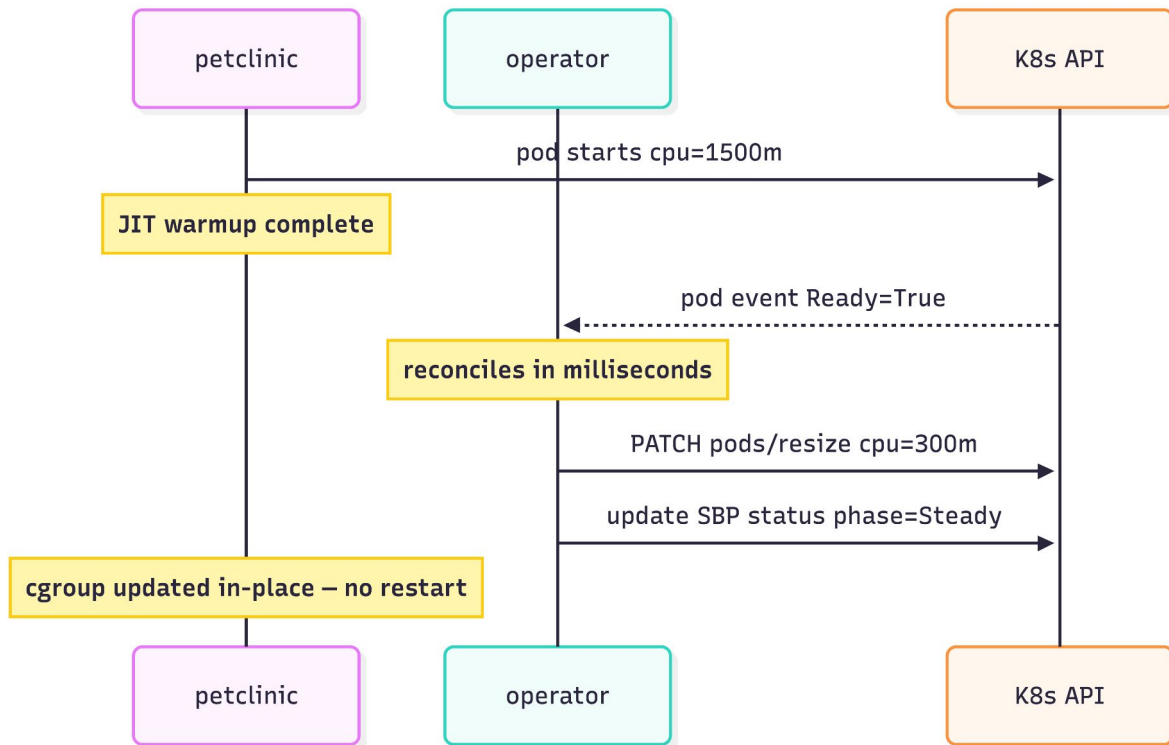
Possible Implementations

Sidecar Pattern vs Deployment

	Sidecar	Deployment
Trigger	Polls from inside the pod	Polls from outside every N s
Resize latency	< 5s	< N s (tunable)
Self-contained	Yes	No
Add a workload	Inject sidecar	Add a label
Infra overhead	1 container per pod	1 shared Deployment
Shrinkage Trap	None	Low

Possible Implementations

Kubernetes Operator



I am running *inside the same pod* as the application. I watch its readiness endpoint every 5 seconds, and the moment it says it is ready I call the Kubernetes API to drop the CPU to 300m. Then I go to sleep — my job is done.

Possible Implementations

Kubernetes Operator

```
apiVersion: startup.boost.io/v1alpha1
kind: StartupBoostPolicy
metadata:
  name: petclinic-boost
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: petclinic
  containers:
    - name: petclinic
      steadyCPU: "300m"
```

startup-boost-crd.yaml

```
# Observable state – first-class Kubernetes resource
kubectl get sbp -n cpu-burst-demo
# NAME                STEADYCPU    AGE
# petclinic-boost     300m         2m
```

```
kubectl describe sbp petclinic-boost -n cpu-burst-demo
# Status:
#   Boosted Pods:
#     Name:          petclinic-xxx
#     Phase:         Steady
#     Resized At:    2026-05-07T...
```

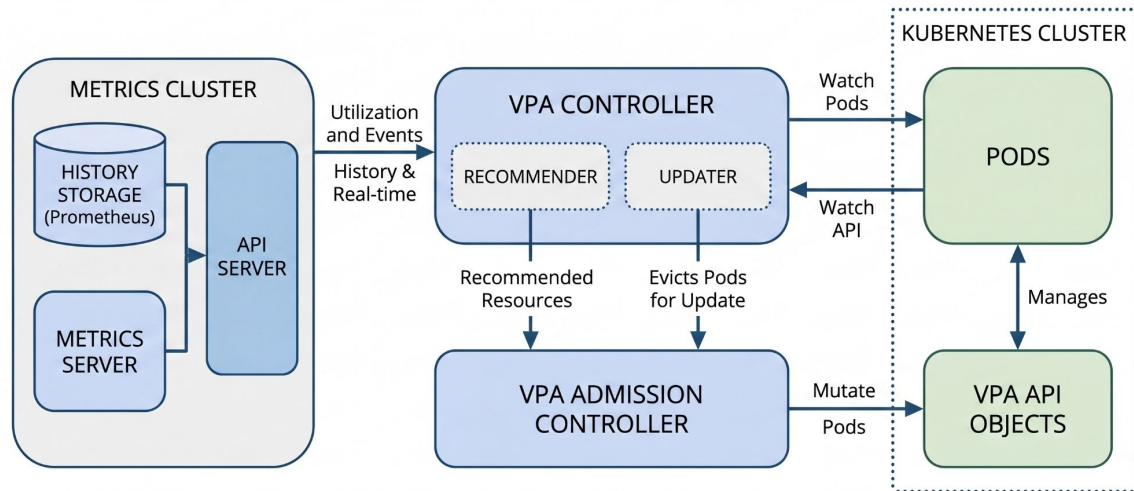
Possible Implementations

Sidecar Pattern vs Deployment vs K8s Operator

	Sidecar	Deployment	Operator
Trigger	Polls from inside the pod	Polls from outside every N s	Pod event — no polling
Resize latency	< 5s	< N s (tunable)	Milliseconds
Self-contained	Yes	No	No
Add a workload	Inject sidecar	Add a label	Create a policy
Infra overhead	1 container per pod	1 shared Deployment	CRD + operator
Shrinkage Trap	None	Low	None

Possible Implementations

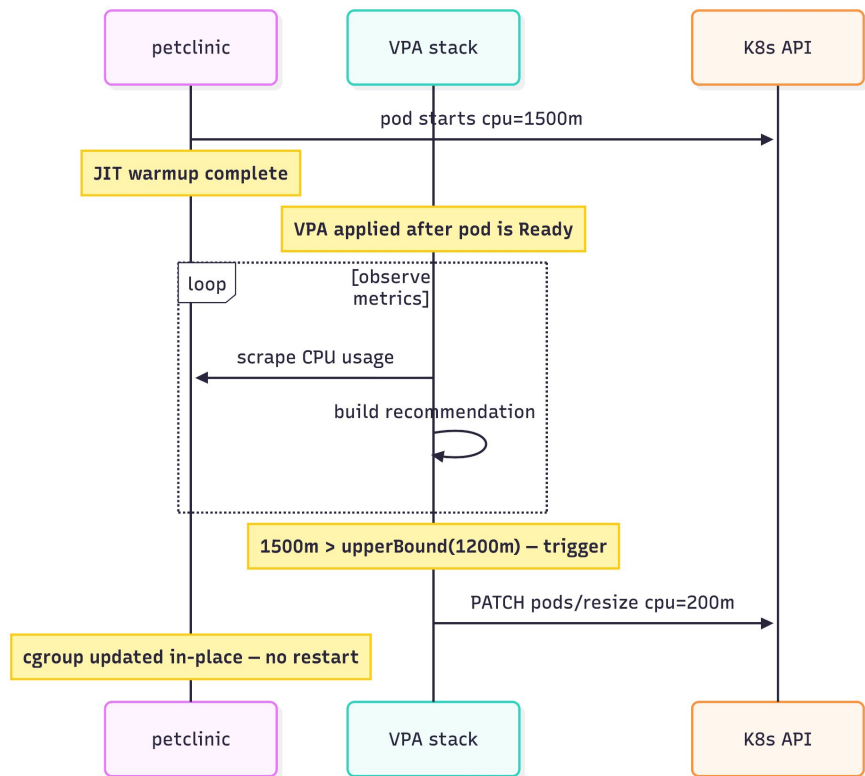
Vertical Pod Autoscaler



VPA is a Kubernetes add-on that watches actual CPU and memory usage and automatically adjusts pod resources to match reality.

Possible Implementations

Vertical Pod Autoscaler



I *observe how much CPU your pod actually uses and build a statistical model over time.*

When the allocation drifts outside my recommended range, I call the Kubernetes API to resize the pod in-place — no restart. I never stop watching.

Possible Implementations

Vertical Pod Autoscaler

```
kind: VerticalPodAutoscaler
spec:
  targetRef:
    kind: Deployment
    name: petclinic
  updatePolicy:
    updateMode: InPlaceOrRecreate
  resourcePolicy:
    containerPolicies:
      - containerName: petclinic
        minAllowed:
          cpu: "200m"
        maxAllowed:
          cpu: "1200m"
```



```
{
  "containerRecommendations": [{
    "containerName": "petclinic",
    "lowerBound": { "cpu": "200m" },
    "target": { "cpu": "200m" },
    "upperBound": { "cpu": "1200m" },
    "uncappedTarget": { "cpu": "35m" }
  }]
}
```

vpa.yaml

Possible Implementations

Sidecar Pattern vs Deployment vs Operator vs VPA

	Sidecar	Deployment	Operator	VPA
Trigger	Polls from inside the pod	Polls from outside every N s	Pod event — no polling	Metrics-based
Resize latency	< 5s	< N s (tunable)	Milliseconds	Minutes
Self-contained	Yes	No	No	No
Add a workload	Inject sidecar	Add a label	Create a policy	Create a VPA object
Infra overhead	1 container per pod	1 shared Deployment	CRD + 1 Deployment	VPA components
Shrinkage Trap	None	Low	None	High

Possible Implementations

VPA: Good to know

AEP-7862: CPU Startup Boost

- [Goals](#)
- [Non-Goals](#)
- [Proposed Design](#)
- [Design Implementation](#)
 - [Work Items](#)
 - [APIs](#)
 - [Validation](#)
 - 1 [AEP-8818: InPlace Update Mode](#) #8818
 - 2 [AEP-7862: Support CPU Startup Boost in VPA](#) #7863
 - 3 [AEP-8026: per-vpa-component-configuration](#) #8026
 - 4 [AEP-8898: Standardize VPA status condition handling](#) #8898
 - [Mitigating Failed In-Place Downsizes](#)
 - [Feature Enablement and Rollback](#)
 - [How can this feature be enabled / disabled in a live cluster?](#)
 - [Kubernetes Version Compatibility](#)
- [Test Plan](#)
- [Examples](#)
 - [Per-pod configurations \(`startupBoost` configured in `VerticalPodAutoscalerSpec` \)](#)
 - [Startup CPU Boost Enabled & VPA Disabled](#)
 - [Startup CPU Boost Disabled & VPA Enabled](#)

VPA AEP statuses

View 1

+ New view

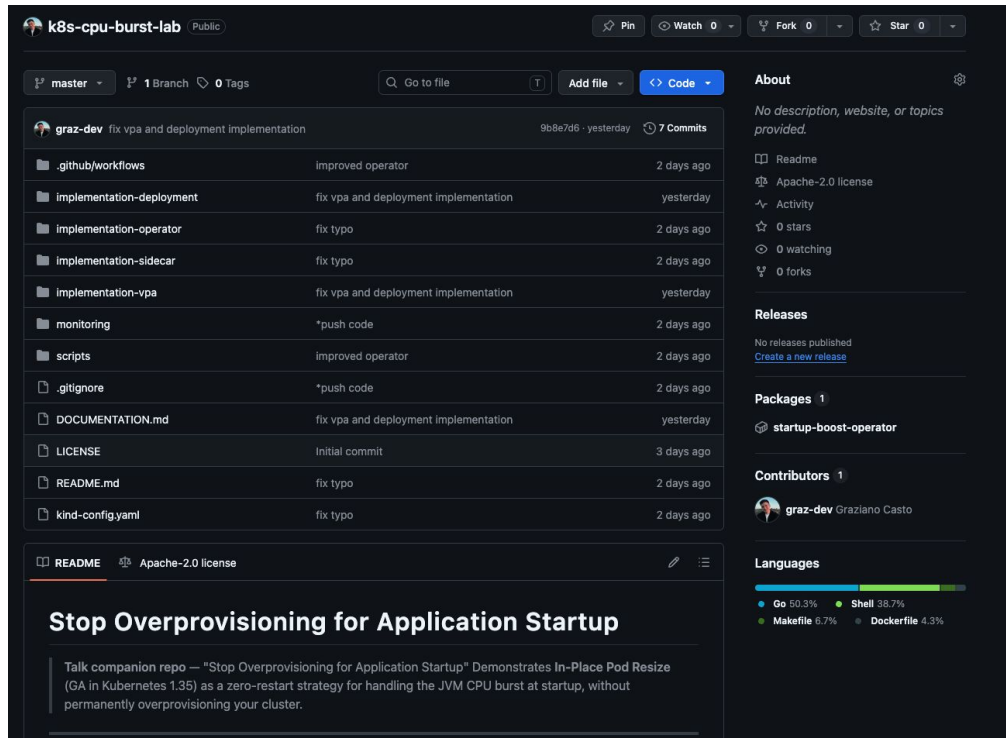
Filter by keyword or by field

	Title	...	Status	⌵⌴2 ...	Since	...	Next	⌵⌴1 ...
1	AEP-8818: InPlace Update Mode #8818		Pre-Alpha	⌵			Alpha	⌵
2	AEP-7862: Support CPU Startup Boost in VPA #7863		Pre-Alpha	⌵			Alpha	⌵
3	AEP-8026: per-vpa-component-configuration #8026		Alpha	⌵	1.5.0		Alpha	⌵
4	AEP-8898: Standardize VPA status condition handling #8898		Uncommitted or new	⌵				⌵



Get the demo!

Try the different implementations by yourself



The screenshot shows the GitHub repository interface for `k8s-cpu-burst-lab` by user `graz-dev`. The repository is public and has 7 commits. The file list includes:

- `.github/workflows`: Improved operator (2 days ago)
- `implementation-deployment`: fix vpa and deployment implementation (yesterday)
- `implementation-operator`: fix typo (2 days ago)
- `implementation-sidecar`: fix typo (2 days ago)
- `implementation-vpa`: fix vpa and deployment implementation (yesterday)
- `monitoring`: *push code (2 days ago)
- `scripts`: Improved operator (2 days ago)
- `.gitignore`: *push code (2 days ago)
- `DOCUMENTATION.md`: fix vpa and deployment implementation (yesterday)
- `LICENSE`: Initial commit (3 days ago)
- `README.md`: fix typo (2 days ago)
- `kind-config.yaml`: fix typo (2 days ago)

The README section is titled "Stop Overprovisioning for Application Startup" and describes a talk companion repo demonstrating in-place pod resize in Kubernetes 1.35 as a zero-restart strategy for handling JVM CPU burst at startup.

The right sidebar shows repository statistics:

- About**: No description, website, or topics provided.
- Releases**: No releases published. [Create a new release](#)
- Packages**: 1 package: `startup-boost-operator`
- Contributors**: 1 contributor: `graz-dev` (Graziano Casto)
- Languages**: Go 50.3%, Shell 38.7%, Makefile 6.7%, Dockerfile 4.3%



github.com/graz-dev/k8s-cpu-burst-lab

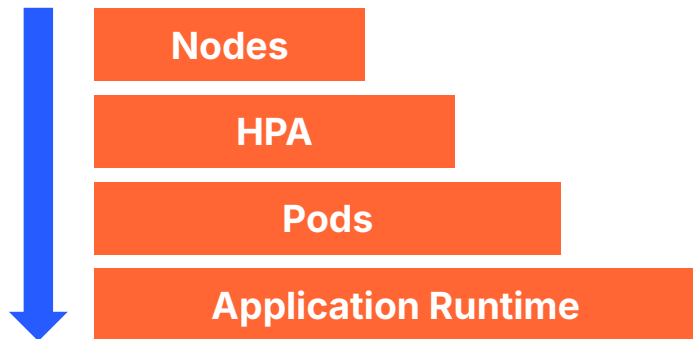
What if I want to reduce the initial spike?

**Well you need to work
on another layer!**



The importance of a full stack approach

Achieving the optimization nirvana



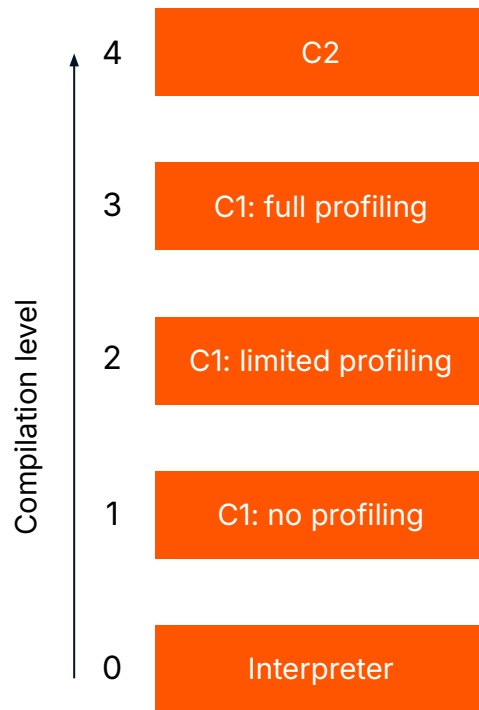
Optimization is a *layered problem*. Working on a single layer could work but to achieve the optimization nirvana you should *consider the entire picture*.

JVM JIT compiler 101

- The JIT compiler compiles bytecode to native code for frequently executed methods ("hotspots")
- The JVM provides two compilers: C1 (client) and C2 (server)
- JIT compilers use CPU/memory to do their work
- Trade-off: code speed vs resource usage vs app runtime
- Key JVM flags
 - -XX:TieredStopAtLevel=N
 - -XX:CompileThresholdScaling=N
 - -XX:-TieredCompilation
 - -XX:C1CompilerCount=N

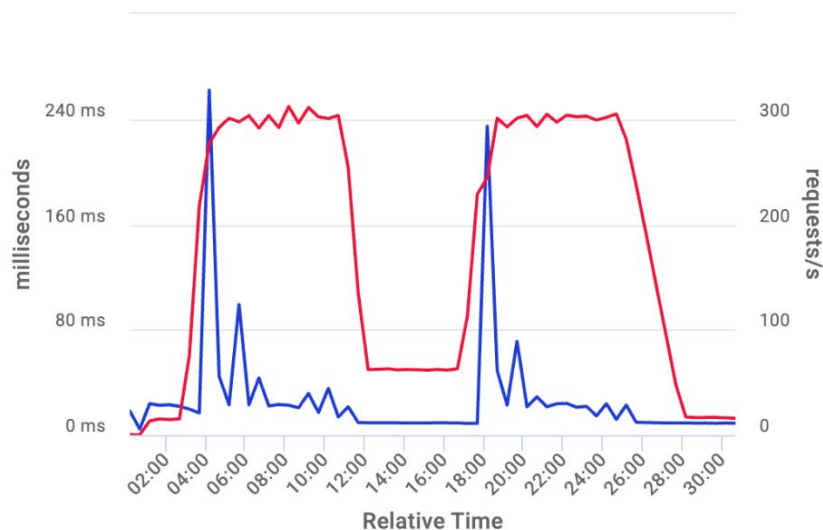


Common Java efficiency issue on K8s

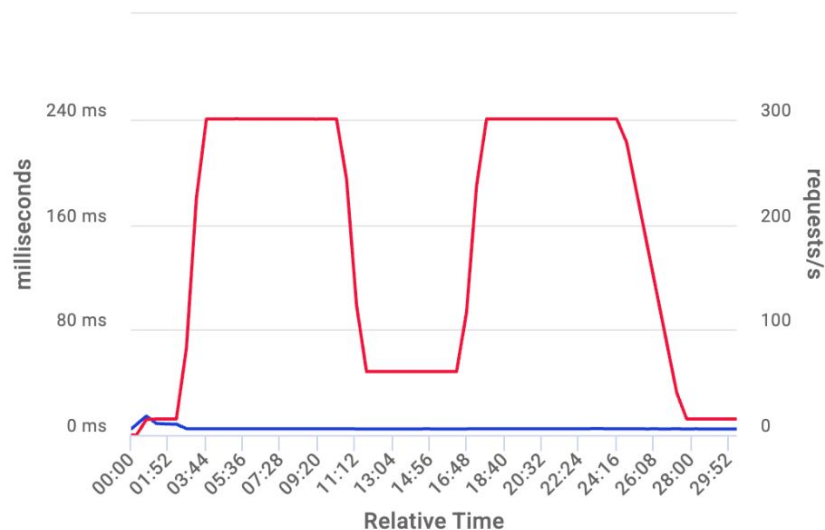


Performance optimization: -76% p90 latency

Before optimization



After optimization

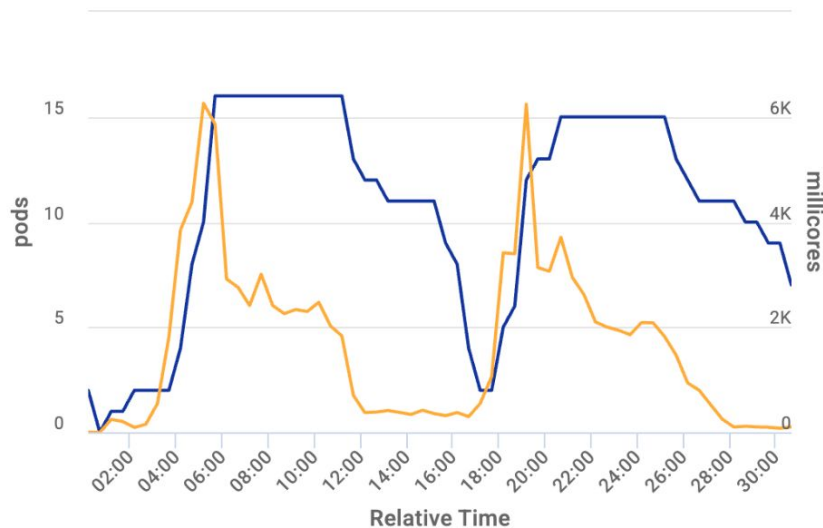


petclinic_webapp.requests_response_time_p90

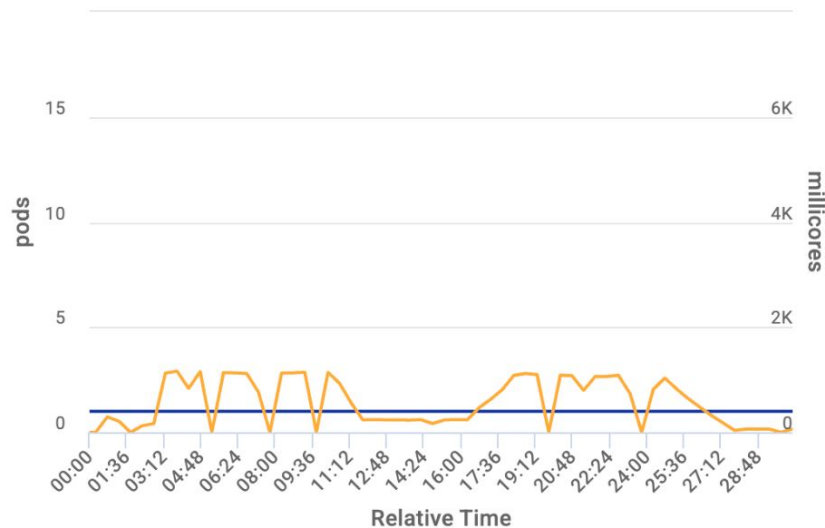
petclinic_webapp.requests_throughput

Performance optimization: -48% CPU used

Before optimization



After optimization



petclinic_workload.k8s_workload_running_pods

petclinic_workload.k8s_workload_cpu_used

Thanks!

Keep in touch and download the slides



castograziano.com/speaking



Casto Graziano



castograziano.com



graziano.casto@akamas.io